

DPI343

Computer Organization and Assembly Language

Lecture 1

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

History of Microprocessors ...

(recall and repeat this topic yourself)



Intel 4004

1971



8008

1972



8086

1978



80386

1985

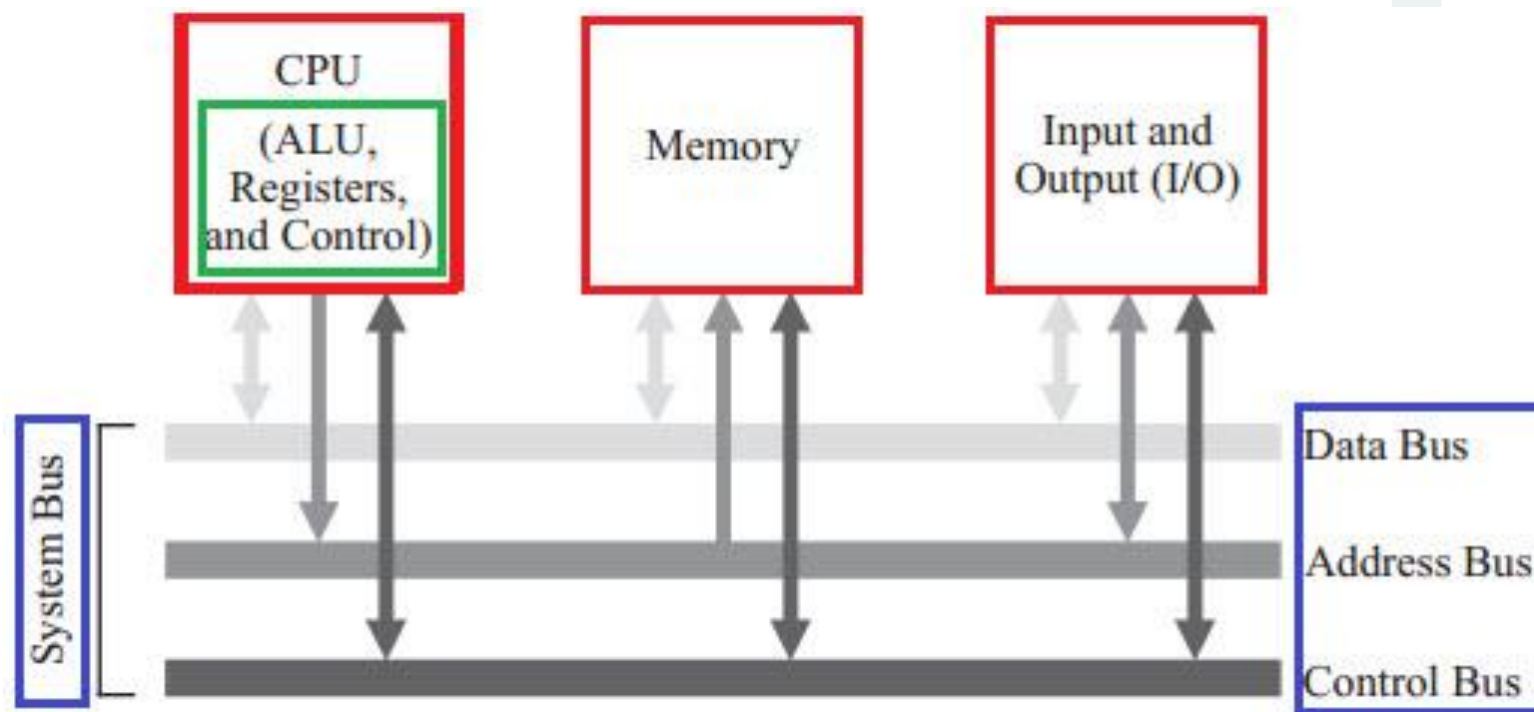


Pentium 1

1993

Von Neumann Computer Architecture

(recall and repeat this topic yourself)



For more, see: http://oopweb.com/Assembly/Documents/ArtOfAssembly/Volume/Chapter_3/CH03-1.html#HEADING1-2

What is an executable code?

- Executable code is loaded into memory by operating system from file (.exe or .dll)
- CPU register EIP always contains the address of the next instruction to be executed by CPU. Hence to start execution operating system loads the address of the entry point into the EIP register
- C++ code to calculate a perimeter of the rectangle:

```
#include <iostream>
```

```
void main() {
```

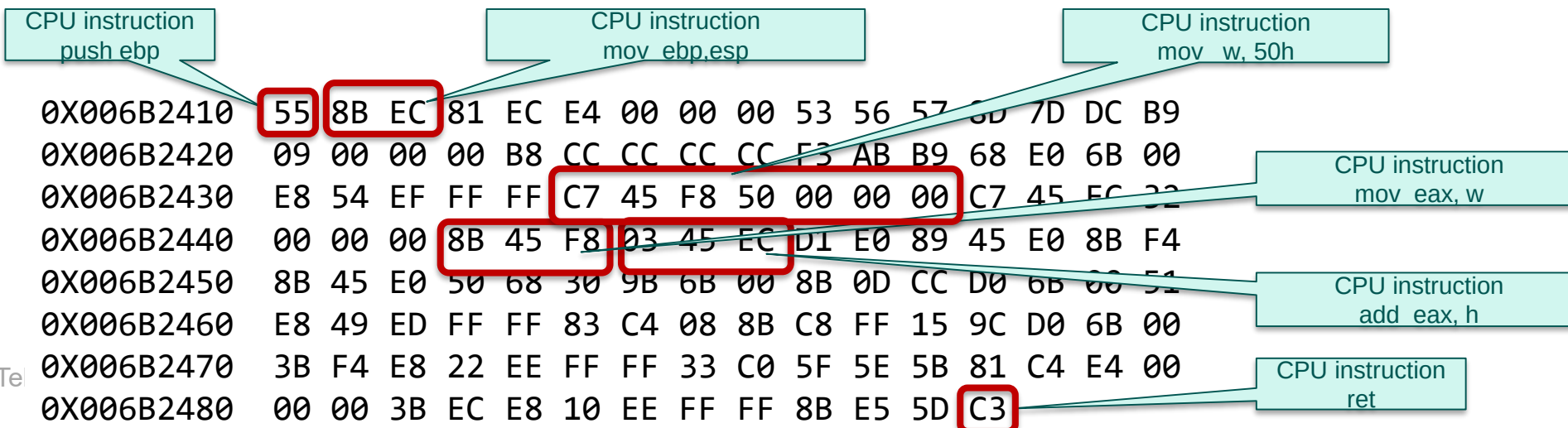
```
    int w=80, h=50, perimeter;
```

```
    perimeter = 2 * (w + h);
```

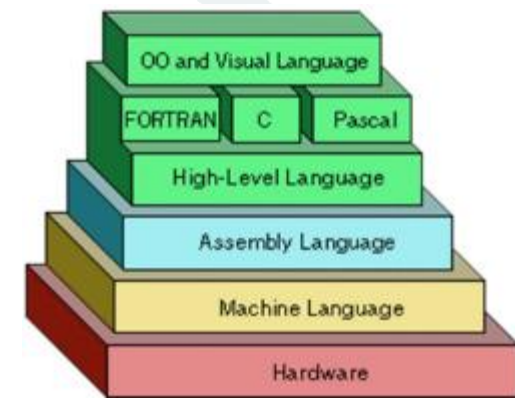
```
    std::cout << "Perimeter is " << perimeter;
```

```
}
```

Executable code of this program in the memory (the CPU register EIP contains 006B2410):



The role of the Assembly language



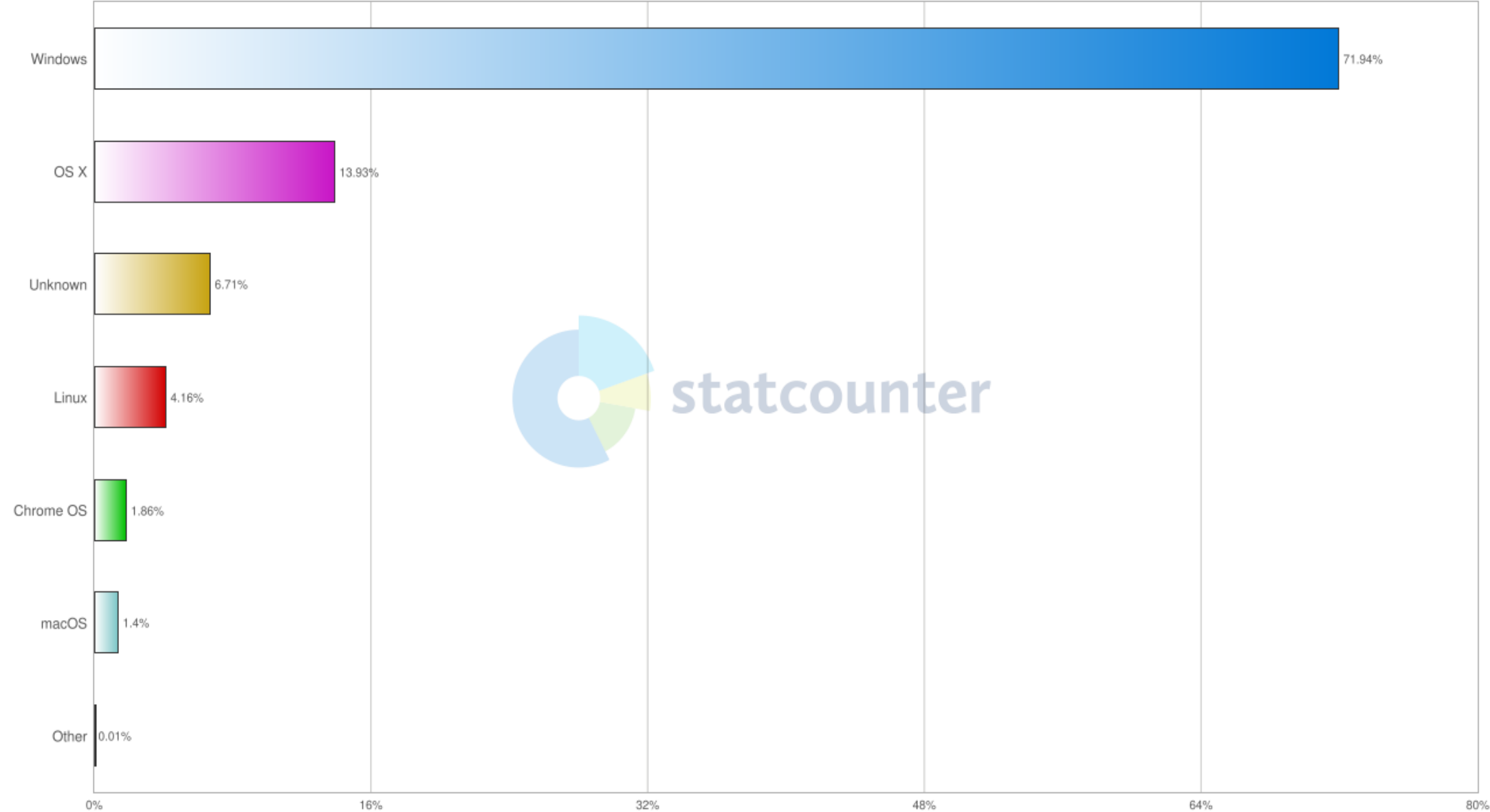
- Closest level to CPU machine code, yet designed for humans.
- Can be used
 - to employ full potential of CPU
 - in understanding how actually CPU works
 - to reverse engineer machine code generated from high level programming languages (e.g., C/C++)
 - to solve "problems" with closed source software – e.g. to overcome high level language compiler quirks or errors
 - to modify closed source software
 - to search for and analyze software for vulnerabilities

x86-64 assemblers

Assembler	Developer	<u>Operating system</u>	<u>FOSS</u>	Development active
<u>FASM</u>	<u>Tomasz Grysztar</u>	<u>DOS</u> , <u>Unix-like</u> , <u>Windows</u>	Yes	Yes
<u>GAS</u>	<u>GNU Project</u>	<u>DOS</u> , <u>OS/2</u> , <u>Unix-like</u> , <u>Windows</u>	Yes	Yes
<u>MASM</u>	<u>Microsoft</u>	<u>DOS</u> , <u>OS/2</u> , <u>Windows</u>	No	Yes
<u>NASM</u>	<u>Simon Tatham</u> , <u>Julian Hall</u> , <u>Hans Peter Anvin</u> , et al.	<u>DOS</u> , <u>Linux</u> , <u>macOS</u> , <u>OS/2</u> , <u>Windows</u>	Yes	Yes
<u>Open Watcom Assembler</u>	<u>Watcom</u>	<u>DOS</u> , <u>FreeBSD</u> , <u>Linux</u> , <u>OS/2</u> , <u>Windows</u>	approved by <u>OSI</u> , but not by <u>FSF</u>	Yes
<u>POASM</u>		<u>Windows</u> , <u>Windows Mobile</u>	No	Yes
<u>TCCASM</u>	<u>Fabrice Bellard</u>	<u>Unix-like</u> , <u>Windows</u>	Yes	Yes
<u>YASM</u> ^[3]		<u>DOS</u> , <u>Unix-like</u> , <u>Windows</u>	Yes	Yes

Desktop Operating System Market Share

StatCounter Global Stats
Desktop Operating System Market Share Worldwide from July 2024 - June 2025



Simple Assembly program for console mode (and useful directives and syntax rules)

```
; Simple Assembly program with no output  
; to calculate perimeter of the rectangle
```

Comment

```
.586
```

Processor selection. Set of Pentium instructions can be used

```
.MODEL flat, stdcall
```

32 bit memory platform, standard calling convention

```
OPTION casemap:none
```

Case sensitive names and labels

```
.DATA
```

Starts data segment

```
w    DWORD    4
```

```
h    DWORD    3
```

```
p    DWORD    ?
```

Allocate and initialize memory fields (variables). The name is used as an address of the first byte of this memory field. Length of this field is defined by the type. ? indicates uninitialized variable

Starts code segment

```
.CODE
```

```
Rect PROC
```

Starts procedure Rect

Comment

```
    mov  eax, w    ;copy 4 bytes from memory address w to the register eax  
    add  eax, h    ;add value stored in 4 memory bytes at address h to the register eax  
    add  eax, eax   ;add the register eax to itself to calculate perimeter as 2*(w+h)  
    mov  p, eax    ;copy the register eax to 4 memory bytes at address p  
    ret           ; return to the caller
```

5 CPU instructions to execute

```
Rect ENDP
```

End of the procedure Rect

```
END Rect
```

End of the source code.
Address of the entry point is Rect.

Compiling and linking

1. Install Visual Studio and COAL Toolkit for 32 bit mode on your computer
2. Create plain text file rectangle.asm which contains Assembly source code
3. Run script VS_masm_x86_cmd.bat to open console window
4. Compile using ml.exe

ml /c rectangle.asm

/c - compile only

recommended:
/Fl<file name> - create a listing file



4. Link using linker link.exe

link /subsystem:console rectangle

Create console mode executable

Default extension .obj is used by linker



5. Run rectangle.exe

rectangle

or use debugger

windbg rectangle

or

ollydbg rectangle

Learn more about compiler and linker options and debugging

ml.exe

<https://learn.microsoft.com/en-us/cpp/assembler/masm/ml-and-ml64-command-line-reference>

link.exe

<https://learn.microsoft.com/en-us/cpp/build/reference/linking>

WinDbg

<https://learn.microsoft.com/en-us/windows-hardware/drivers/debuggercmds/windbg-overview>

COAL Toolkit scripts

For more convenient work in command line mode,
you should use scripts you'll find in the COAL Toolkit for 32 bit mode (see description in course page):

- bc.bat – builds a console application from a source .asm file
- bcd.bat – builds a console application from a source .asm file and runs it in WinDbg
- d.bat – runs the executable in the debugger WinDbg
- bw.bat – builds a Windows application from a source .asm file
- bcdolly.bat – builds a console application from a source .asm file and runs it in ollydbg
- dolly.bat – runs the executable in the debugger ollydbg

Simple Assembly program for Windows mode

```
; Simple Windows dialog app to say Hello
```

```
.586
```

```
.model flat, stdcall
```

```
option casemap:none
```

```
include windows.inc
```

```
includelib user32.lib
```

```
includelib kernel32.lib
```

```
extern MessageBoxA@16:PROC
```

```
extern ExitProcess@4:PROC
```

```
.stack 4096
```

```
.const
```

```
Caption byte "My First Windows Program",0
```

```
Text byte "Hello!",0
```

```
.code
```

```
WinMain PROC
```

```
    push MB_OK + MB_ICONEXCLAMATION
```

```
    push offset Caption
```

```
    push offset Text
```

```
    push 0
```

```
    call MessageBoxA@16
```

```
    push 0
```

```
    call ExitProcess@4
```

```
WinMain ENDP
```

```
end WinMain
```

Case sensitive names and labels are strongly recommended

Include files to enable use of Windows API functions and libraries

Define external names of API functions

Stack segment (size 4096 bytes)

Read-only data segment

Assembly directives and instructions are not case sensitive, only names are

Values of 4 parameters are pushed onto stack and then the CPU instruction call is used to transfer control to API function.

It will be explained in details in Lecture 6.

It is recommended that you always return from your code to Windows using this API function. Value of the parameter is return code

Compiling and linking for Windows mode

- Call compiler and linker from command line

ml /c hellowin.asm

link /subsystem:windows hellowin

hellowin

Create the executable for Windows mode

All libraries (.lib files) requested in include directives must be available for linking

- or use COAL Toolkit script bw.bat:

bw hellowin

hellowin

Explore script content that comes with COAL Toolkits and learn more about Windows commands used in scripts

<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands>

Using Visual Studio for pure Assembly source code

- Create and customize a project for pure Assembly source code
 - Create Empty project (set project name and choose a folder to store your projects)
 - Right click on project name in the left pane, select **Build dependencies->Build Customizations** and check **masm**
 - Right click on project name in the left pane, select **Properties**, select **Linker->System->Subsystem** and select **Console (/SUBSYSTEM:CONSOLE)**
 - add source file with extension .asm
 - check properties of the source file to ensure it is included in the build
 - select **Solution Platform x86**
 - if external library functions are used, make sure these libraries are listed in project **Properties**: select **Linker->Input->Additional Dependencies** and add a list of libraries needed
 - build and run the program or use built-in debugger

See also hints at:

www.youtube.com/watch?v=-fCyvipptZU

<https://resources.infosecinstitute.com/assembly-programming-visual-studio-net>

Inline Assembly in C++

- The inline assembler lets you embed assembly instructions in C or C++ source without extra assembly and link steps.
- The inline assembler is built into C/C++ compiler.
- Inline assembly code can use any C or C++ variable or function name that is in scope.
- Assembly code can be mixed with C and C++ statements.
- The **__asm** keyword (two underscore symbols and asm) invokes the inline assembler and can appear wherever a C or C++ statement is legal.
- **__asm** keyword must be followed by an assembly instruction, or a group of instructions enclosed in braces:

```
__asm{  
    ; Assembly instructions  
}
```

```
__asm mov eax, -1 ; single instruction
```

See Microsoft manual for more information (incl. some restrictions)

<https://learn.microsoft.com/en-us/cpp/assembler/inline/inline-assembler>

Inline Assembly in C++. Example

```
#include <iostream>
using namespace std;
void main()
{ //Calculate perimeter of the rectangle

    int w = 100, h = 20;      ; doubleword format integers
    int p;

    _asm{
        mov  eax, w
        add  eax, h          ; w+h
        add  eax, eax        ;calculate perimeter as 2*(w+h)
        mov  p,  eax
    }
    cout << "Assembly result is " << p << endl;

    p = 2*(w+h);
    cout << "C++ result is " << p << endl;
}
```

Assembly instructions

Executable instructions (statements) to be executed by CPU.

The compiler creates executable code for each instruction.

Instruction format:

```
[Label:] [Operation] [Operands] [;Comment]
```

Each field must be separated from other fields by a space or tab.

Examples:

```
    xor eax, eax    ; perform Boolean XOR and store result in EAX
    mov ecx, 10     ; load value 10 in the ECX register
c:  inc eax         ; increment the EAX register
    loop c          ; perform loop instruction (will be explained in Lecture 5)
```

- Typical notation for instructions is to have the destination as the first operand and the source as the second. But there are some exceptions to this rule.
- The operand list may contain memory references, registers, and constant values. Each instruction allows only certain types of operands to appear at each position in the operand list.
- Only one of the operands may be a memory reference, all other must be registers or constant values (but there are some exceptions to this rule).

Assembly directives

Directives - direct the Assembly compiler to do something

- Directives do not generate any executable code
- Data definition directives generate code of initializing data depending on data format

Processor selection directives defines set of instructions to use: .8086, .186, .286, .386, .486, .586, .686, etc.

Simplified segment directives indicates use of memory segments: .CODE, .CONST, .DATA, .STACK, .MODEL, etc.

Directive EQU: assigns value of the expression to name. Syntax: *name EQU expression*

The name can then be used in the code and will be replaced by a value by the compiler (similar to #define in C++)

Example: Size EQU 1024

Comment block directive. Syntax: COMMENT *delimiter* [*text*]

[*text*]

[*text*] *delimiter*

Delimiter can be any symbol not used in the source code.

END directive: marks the end of a module and sets the program entry point to *name*. Syntax: END [*name*]

Assembly data definition directives

Allocates and optionally initializes bytes of memory for each *initializer*.

[name] type initializer [, initializer ...]

Type	Size (bytes)	Comments
BYTE DB	1	BYTE and DB are synonyms
SBYTE	1	Signed value
WORD DW	2	WORD and DW are synonyms
SWORD	2	Signed value
DWORD DD	4	DWORD and DD are synonyms
SDWORD	4	Signed value
QWORD QW	8	QWORD and DQ are synonyms

Examples:

Alpha BYTE 41h

Beta WORD 1, 257, -2

Msg BYTE "Done!", 0

Char DB 'A', 'B', 'C', 'D', '0', '1', '2', '3'

Alpha

41

Beta

Beta+1

Beta +2

Beta+3

Beta+4

Beta+5

01	00	01	10	FE	FF
----	----	----	----	----	----

Msg

Msg+1

Msg+2

Msg+3

Msg+4

Msg+5

44	6F	6E	65	21	00
----	----	----	----	----	----

Char

Char+1

Char+2

Char+3

Char +4

Char+5

Char+6

Char+7

41	42	43	44	30	31	32	33
----	----	----	----	----	----	----	----

Common numeral systems

- Decimal (base 10)
- Binary (base 2)
- Hexadecimal (base 16)
- Octal (base 8)
- To indicate the base of a integer value in Assembly, use the specifier **h** for hexadecimal, **b** for binary, or **o** for octal at the end of the number. Specifiers and hexadecimal digits can be uppercase or lowercase.
- Hexadecimal values must always start with a decimal digit 0–9. If the first digit is letters (A to F), use leading 0 to distinguish it from a symbolic name
- Examples:
 - $21 = 21d = 15h = 10101b = 25o$
 - $0A1h = 161 = 10100001b = 241o$

Leading zero, because otherwise A1h will be considered an identifier!

How to convert integers from decimal to binary and vice versa?

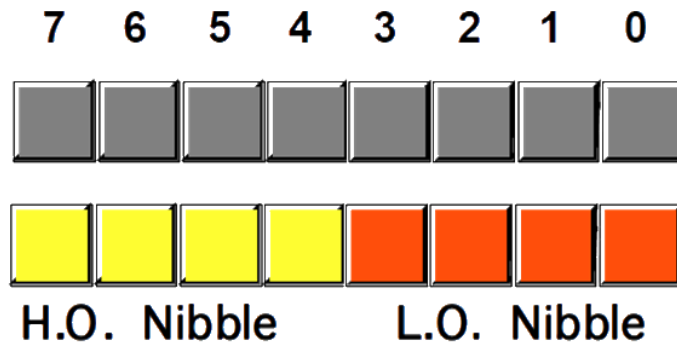
- 1. Convert decimal value to hexadecimal** – divide by 16 and write remainders:
 $93 : 16 = 5$, remainder 13 (D); $5 : 16 = 0$, remainder 5 ; i.e. $93d = 5Dh$
- 2.** Replace each hexadecimal digit with combination of four bits (0 = 0000, 1 = 0001, 2 = 0010, ..., E = 1110, F = 1111):
 $5 = 0101b$, $D = 1101b$; i.e. $5Dh = 0101\ 1101b$
- 3. Convert hexadecimal value to decimal** – calculate using powers of base 16:
 $2BAh = 2 \cdot 16^2 + 11 \cdot 16^1 + 10 \cdot 16^0 = 2 \cdot 256 + 11 \cdot 16 + 10 \cdot 1 = 512 + 176 + 10 = 698$
- 4. The two's complement is used to store negative numbers.**
The high order bit indicates the sign (0 – positive, 1 – negative).
One signed byte $11111110b = -2d$
To find two's complement convert all ones to zeros and add 1:
 $2 = 00000010b$; $11111101b + 1b = 11111110b$
- 5.** A faster way to convert negative decimal to hexadecimal: first, convert the positive value then subtract from hexadecimal zeros with one extra high order digit 1:
 - given decimal value -12
 - positive hexadecimal 0Ch
 - for word representation subtract 10000h

$$\begin{array}{r} - 000Ch \\ \hline FFF4h \end{array}$$

Computer data types.

Byte

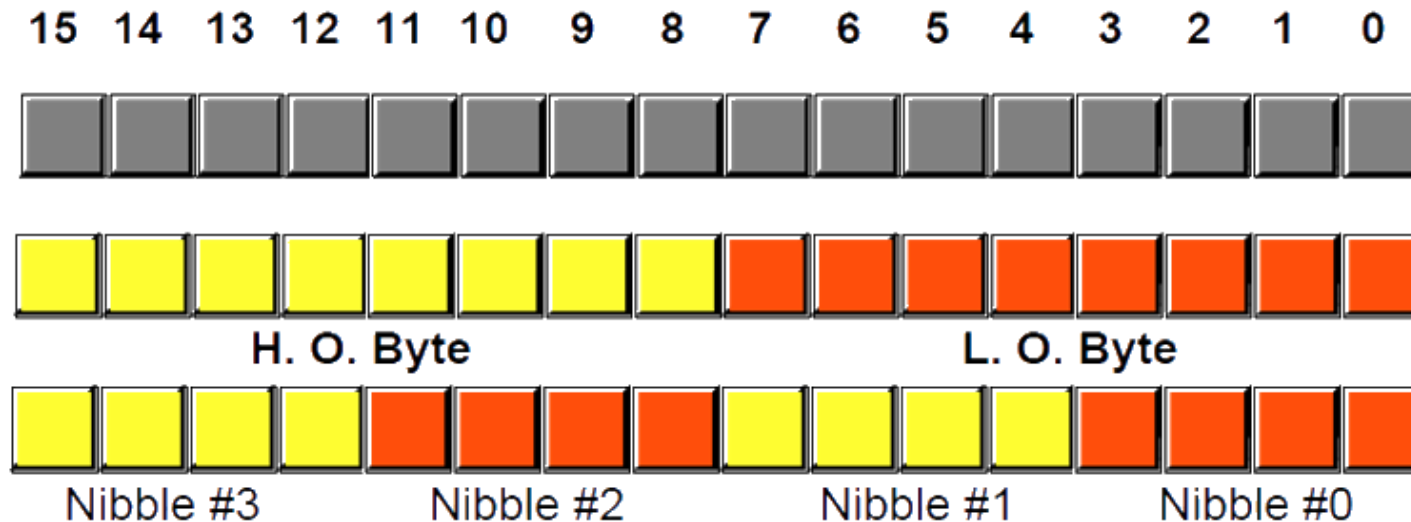
- One byte – 8-bit sequence
- Byte can represent 256 different values (2^8)
- Unsigned range: $0 \dots FF_{16}$ ($0 \dots 255_{10}$)
- Signed range: $80_{16} \dots 7F_{16}$ ($-128_{10} \dots 127_{10}$)



Computer data types.

Word

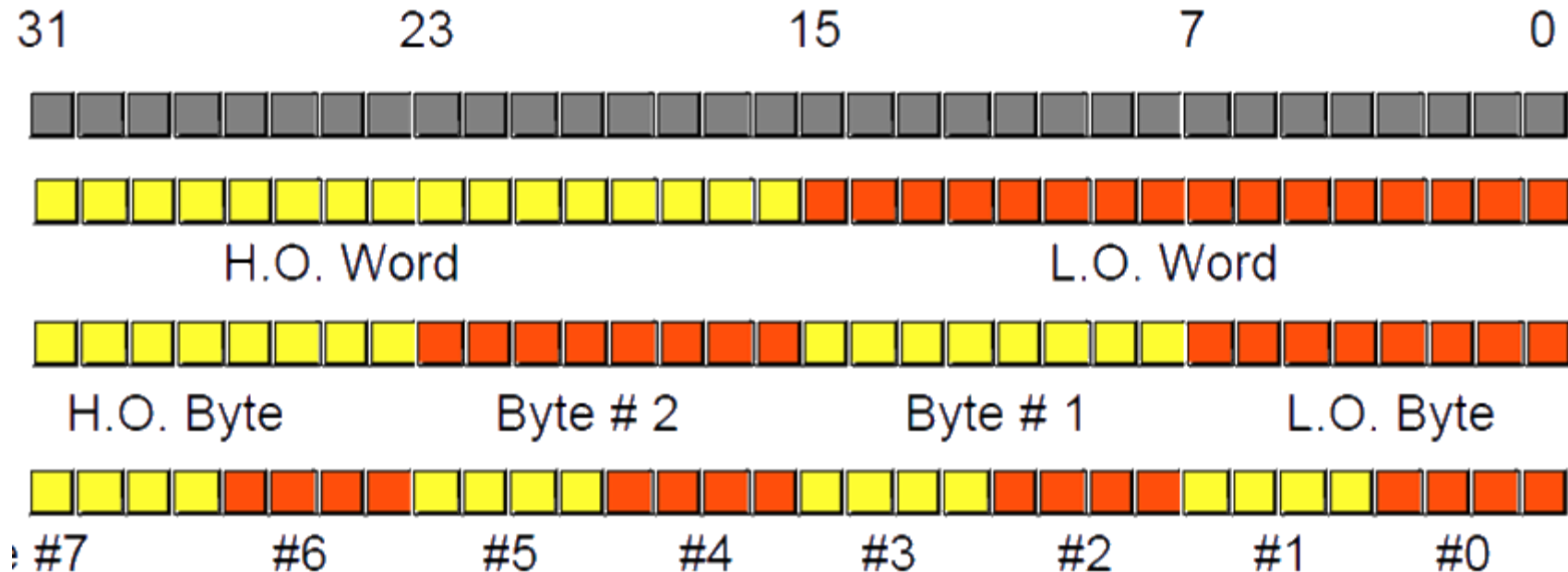
- One word – 16-bit sequence (i.e., two bytes)
- Word can represent 65536 different values (2^{16})
- Unsigned range: $0 \dots \text{FFFF}_{16}$ ($0 \dots 65,535_{10}$)
- Signed range: $8000_{16} \dots 7FFF_{16}$ ($-32,768_{10} \dots 32,767_{10}$)



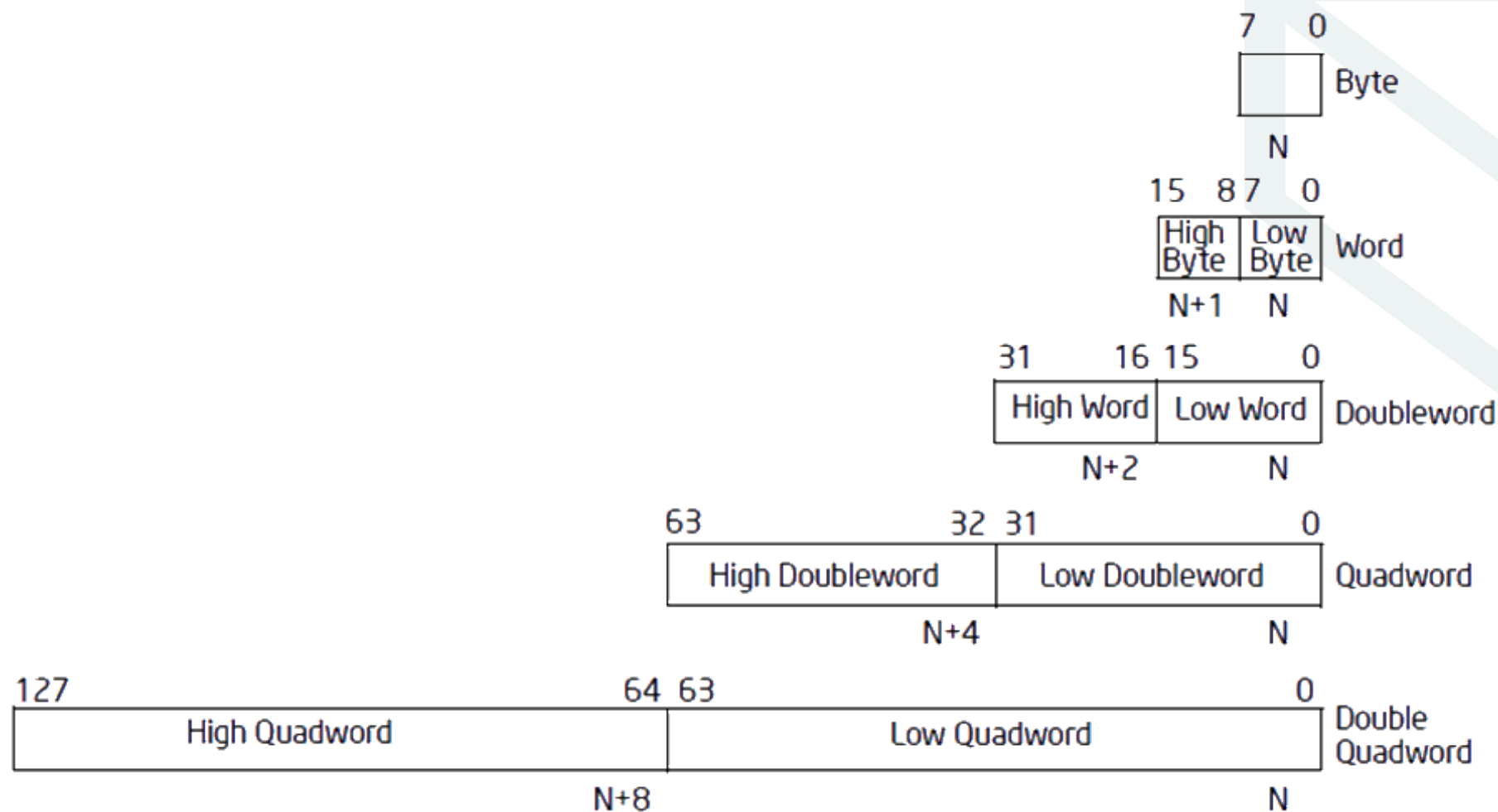
Computer data types.

Doubleword

- One doubleword – 32-bit sequence (2 words / 4 bytes)
- 2^{32} different values
- Unsigned range: $0 \dots \text{FFFF'FFFF}_{16}$ ($0 \dots 4,294,967,295_{10}$)
- Signed range: $8000,0000_{16} \dots 7\text{FFF,FFFF}_{16}$ ($-2,147,483,648_{10} \dots 2,147,483,647_{10}$)



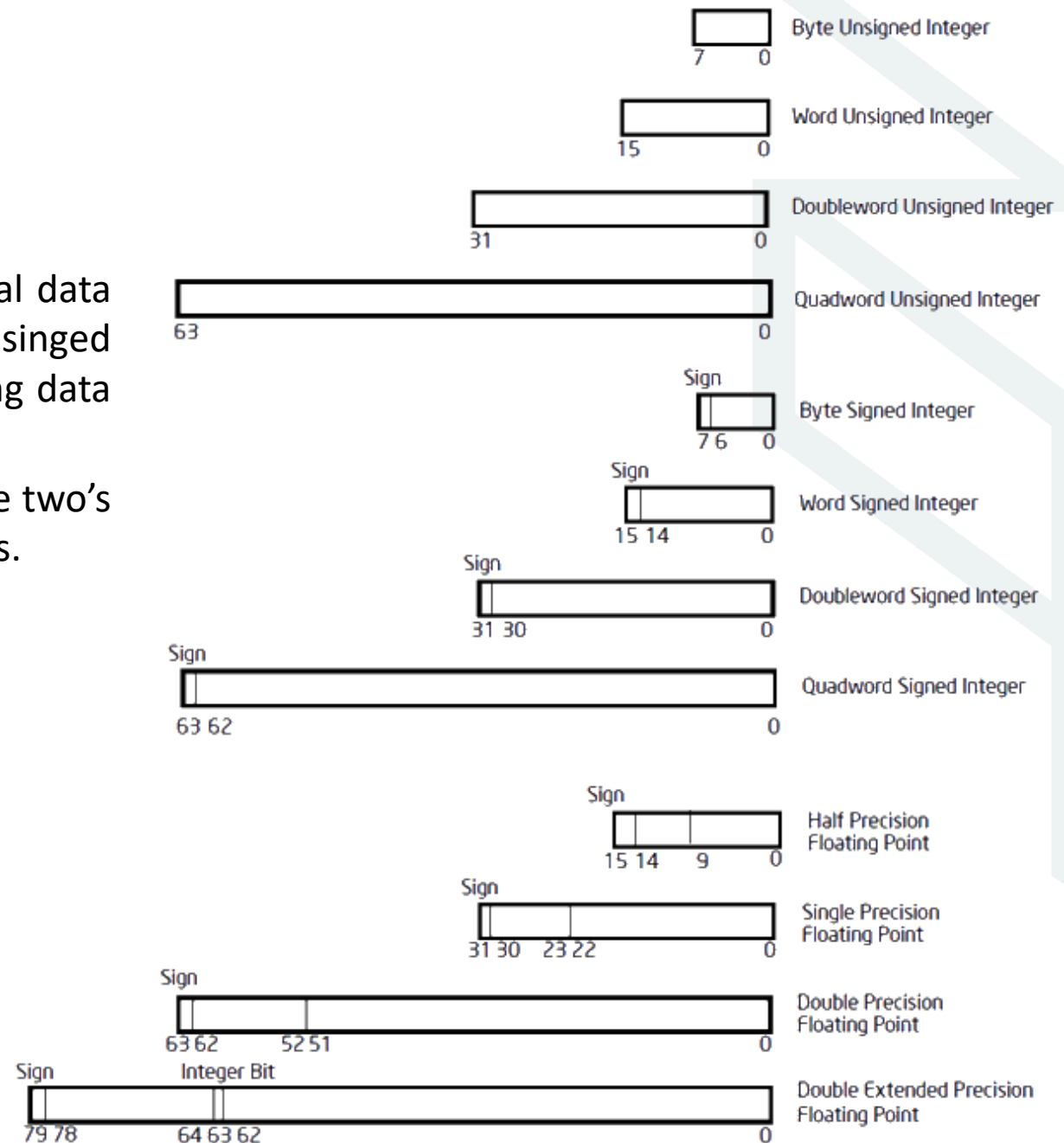
Summary of IA-32 data types



Note: The quadword was introduced into the IA-32 architecture in the Intel486 processor; the double quadword was introduced in the Pentium III processor with the SSE extensions. For more, see: Intel vol1 section 4.1

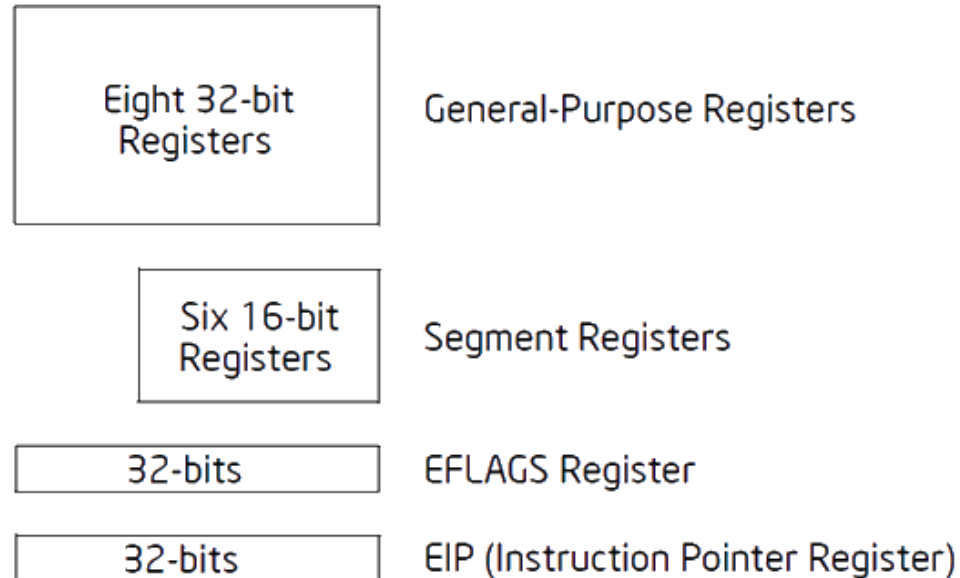
Numeric data types

- Based on the fundamental data types – various integer (signed and unsigned) and floating data types can be defined.
- Signed integers always are two's complement binary values.

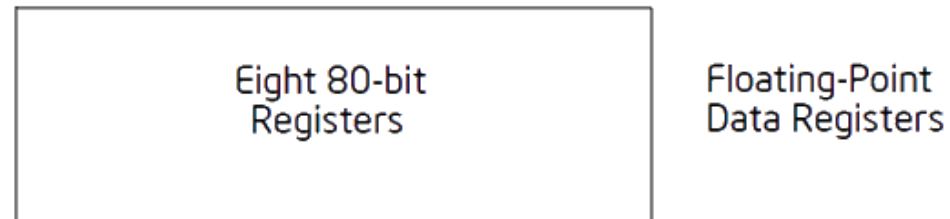


IA-32 CPU execution environment (simplified)

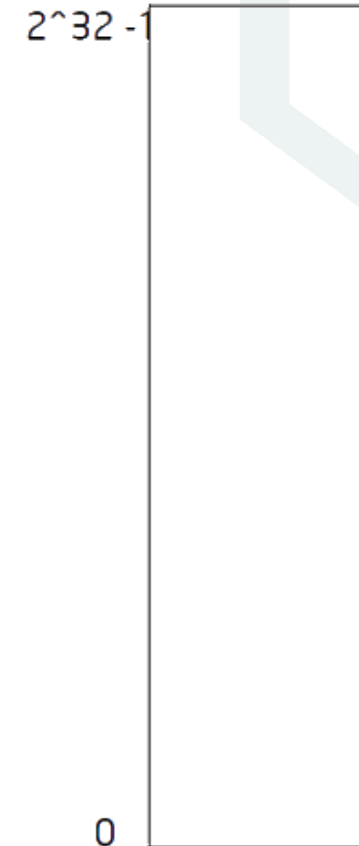
Basic Program Execution Registers



FPU Registers



Address Space*



IA-32 general purpose registers

31	16 15	8 7	0	16-bit	32-bit
	AH	AL		AX	EAX
	BH	BL		BX	EBX
	CH	CL		CX	ECX
	DH	DL		DX	EDX
	BP				EBP
	SI				ESI
	DI				EDI
	SP				ESP

General purpose register mapping



Example:

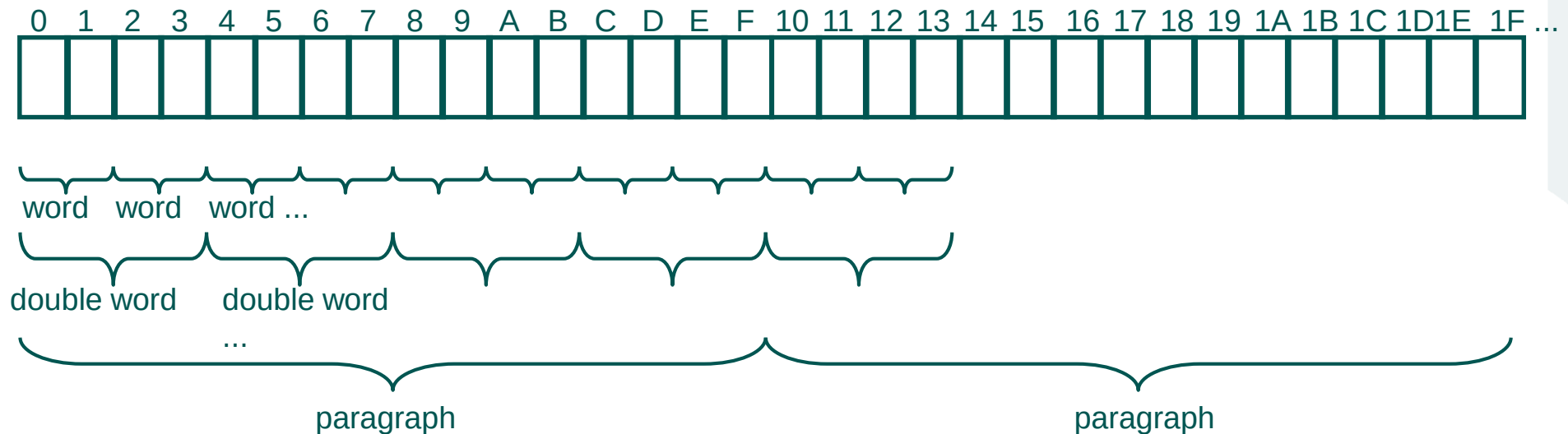
- Register EAX (binary) :

0000'0000'0000'0000'0001'0100'1100'0011

- EAX (in hex) = ? **000014C3** **decimal $1*4096+4*256+12*16+3 = 5315$**
- AX (in hex) = ? **14C3**
- AH (in hex) = ? **14** **decimal $1*16 + 4 = 20$**
- AL (in hex) = ? **C3** **unsigned value is decimal 195**
 signed value is decimal -61

Computer memory addressing

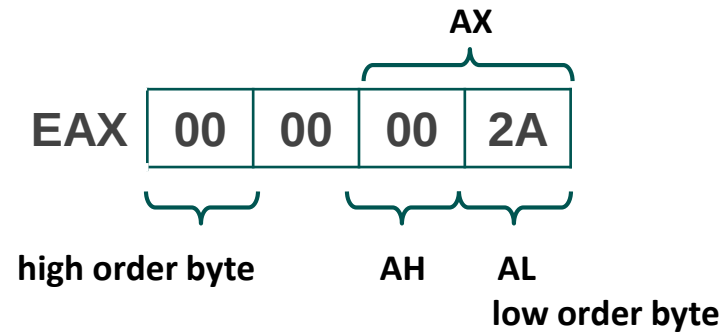
- RAM
- ROM
- Virtual memory
- Memory units
 - Bit, Byte, Word, Doubleword, Quadword, Paragraph



Representation of integer values

Copy integer value 42 (decimal) to the register EAX

```
mov  eax, 42
```

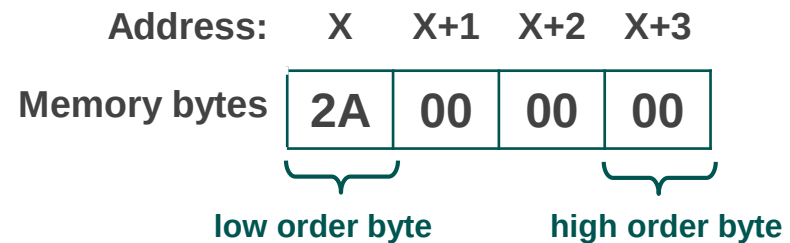


Copy value from the register EAX to the memory doubleword X

```
X  dword 0
```

...

```
mov  X, eax
```

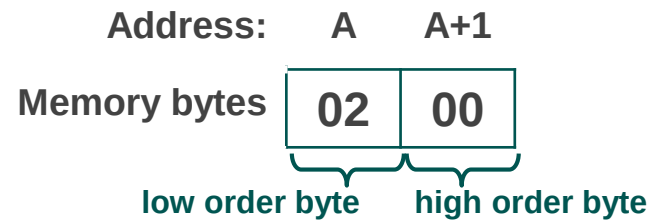


Representation of integer values

Define memory field containing value 2

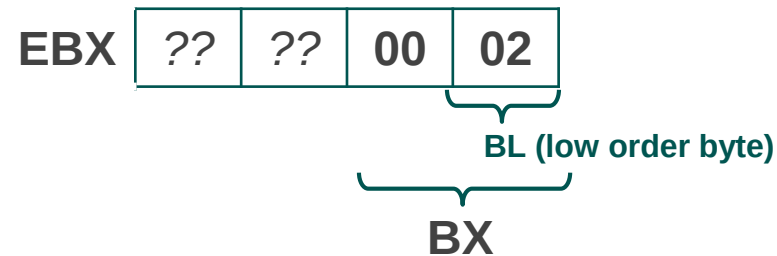
A word 2

Name A represents address of the first memory byte of this memory field with the smallest address and hence this is low order byte of this numeric value.



Copy two bytes from memory address A to the register BX

mov bx, A



Add -4 to the register BX

add bx, -4



Special roles of CPU registers

General purpose registers

- EAX — **Accumulator** for operands and results data
- EBX — Pointer to **base** of data in the memory data segment
- ECX — **Counter** for loop and string operations
- EDX — **Data** for multiplication/division, I/O pointer

Index registers

- ESI — Pointer to data in the memory data segment; **source index** for string operations
- EDI — Pointer to data (or destination) in the memory data segment; **destination index** for string operations

Stack registers

- ESP — **Stack pointer** (in the stack segment)
- EBP — **Base pointer** for the stack operations in the stack segment

Thank You!

